Quad-Dimensional DataMatrix

Request for Discussion: Estados GIF Hexadecimais e Sequências Mantidas no App

João Paulo De Luca Jr Jorge Luiz Espanhol

4 de novembro de 2026 · RfD 0.2.0

Resumo

Quad-Dimensional DataMatrix é proposto como estrutura de discussão para dados hexadecimais representados por estados GIF indexados. O perfil de trabalho usa um canvas de 256 por 256 pixels, 64 por 64 módulos efetivos, até 64 entradas hexadecimais na paleta e até 64 frames. Quatro coordenadas lógicas identificam posição horizontal, posição vertical, estado temporal e dimension. Seu catálogo cartesiano máximo contém 16.777.216 tuplas candidatas. Uma sequência densa seleciona uma entre 64 alternativas em cada módulo e, assim, transporta até 192 KiB de payload bruto em um ciclo de 64 frames, nominalmente oito segundos a 8 fps. Uma sequência mantida no app oferece uma segunda interação: avançar na cadência escolhida, pausar por seleção do usuário e copiar o código hexadecimal do estado para um serviço receptor. O paper desenvolve o modelo de contagem, separa os dois papéis do tempo e examina mapeamentos um a um entre Unicode e hexadecimal como candidatos a um futuro dicionário ou chave. Um estudo de 16 frames e um visualizador local bilíngue acompanham a discussão. A seção Escopo e limitações concentra as qualificações. O documento convida ao refinamento antes da proposta de uma especificação.

1 Propósito e status da discussão

Quad-Dimensional DataMatrix é um Request for Discussion (RfD) de **João Paulo De Luca Jr e Jorge Luiz Espanhol**. Reúne um catálogo de quatro coordenadas, transporte GIF indexado, valores hexadecimais e seleção de estados mantidos no app.

O objetivo imediato é tornar os objetos propostos contáveis e testáveis. O leitor deve conseguir identificar o conteúdo de um estado, como o app o seleciona, o que o usuário copia e qual interpretação de um valor hexadecimal está sendo proposta. Uma etapa posterior poderá desenvolver uma especificação a partir da discussão.

O vocabulário de trabalho é:

Termo	Significado neste RfD
Pixel armazenado	Um índice na imagem armazenada
Módulo efetivo	Bloco de 4×4 com índice repetido
Dimension	Identificador ativo da paleta hexadecimal
Estado	Matriz completa em um índice temporal
Sequência	Conjunto ordenado de até 64 estados
Tupla do catálogo	Tupla candidata (x, y, t, d)
Código	Valor associado a um estado para consumo
Chave	Entrada de dicionário ou segredo proposto à discussão

Inglês é o idioma padrão do repositório e do manuscrito. Uma edição completa em português é mantida em paralelo. As qualificações estão concentradas em Escopo e limitações; o argumento a

seguir desenvolve diretamente o modelo de trabalho.

2 Perfil GIF de trabalho e valores hexadecimais

2.1 Geometria e entradas ativas

O perfil atual é descrito por

$$W_p = H_p = 256, \qquad s = 4, \qquad W = H = 64, \qquad K \leq 64, \qquad F \leq 64.$$

O frame armazenado possui $W_p H_p = 65.536$ pixels. Sua quantidade de módulos independentes é

$$WH = \frac{W_p H_p}{s^2} = 4.096.$$

O fator $s^2 = 16$ repete cada índice selecionado em um módulo quadrado. Determina a geometria de renderização, enquanto WH determina a quantidade de escolhas independentes de módulo.

Cada entrada ativa da paleta é um valor hexadecimal de seis dígitos, como #`12ABEF`. Para o estado t, o mapeamento é

$$P_t : \{0, \ldots, K-1\} \longrightarrow \{\#\texttt{000000}, \ldots, \#\texttt{FFFFFF}\}.$$

A relação pública é

$$\text{dimension} \longleftrightarrow \text{índice da paleta} \longleftrightarrow \text{valor hexadecimal}.$$

O adaptador atual trata a paleta ordenada como dicionário de código. Sua ordem pode mudar entre estados, preservando a representação hexadecimal de cada entrada. GIF89a fornece o contêiner de tabelas indexadas e blocos de imagem.

2.2 Representação e custo da tabela

Um valor hexadecimal de seis dígitos possui domínio inteiro de 24 bits:

$$16^6 = 2^{24} = 16.777.216.$$

O alfabeto ativo tem K entradas; escolher uma entrada custa $\log_2 K$ bits no modelo de contagem sem restrições. Com $K = 64$, essa escolha transporta seis bits.

Uma tabela GIF com K entradas ocupa

$$B_{\text{table}} = 3K \text{ bytes}.$$

Com 64 entradas, a tabela ocupa 192 bytes. Se cada estado fornecer sua própria tabela, a alocação será de $3KF$ bytes. Descrições textuais em hexadecimal e o contêiner codificado possuem seus próprios orçamentos de armazenamento. O modelo mantém essas grandezas observáveis separadamente.

3 O catálogo de quatro coordenadas

3.1 Catálogo cartesiano e estado selecionado

Sejam

$$X = \{0, \ldots, W-1\}, \quad Y = \{0, \ldots, H-1\}, \quad T = \{0, \ldots, F-1\}, \quad \Delta = \{0, \ldots, K-1\}.$$

O catálogo proposto de quatro coordenadas é

$$\Omega = X \times Y \times T \times \Delta, \qquad |\Omega| = WHFK.$$

Para o perfil máximo,

$$|\Omega| = 64^4 = 16.777.216.$$

Uma sequência densa escolhe uma dimension em cada posição e estado temporal:

$$D : T \times Y \times X \longrightarrow \Delta.$$

De maneira equivalente, define-se um tensor indicador de quatro índices

$$A[t, y, x, d] = \begin{cases} 1, & D[t, y, x] = d, \\ 0, & \text{caso contrário.} \end{cases}$$

A condição de seleção é

$$\sum_{d=0}^{K-1} A[t, y, x, d] = 1.$$

O catálogo enumera todas as tuplas candidatas; a sequência densa seleciona uma dimension para cada uma de suas WHF posições. Isso dá significado preciso às quatro coordenadas lógicas do projeto.

3.2 Um endereço hexadecimal do catálogo

Com os quatro intervalos de coordenadas limitados a 64, um endereço do catálogo em ordem de linhas pode ser escrito como

$$a = (((64t + y)64 + x)64 + d).$$

Seu intervalo é

$$0 \leq a \leq 64^4 - 1 = \texttt{0xFFFFFF}.$$

A inversão é

$$d = a \bmod 64, \qquad x = \lfloor a/64 \rfloor \bmod 64,$$

$$y = \lfloor a/64^2 \rfloor \bmod 64, \qquad t = \lfloor a/64^3 \rfloor.$$

Um endereço usa 24 bits, com seis bits para cada coordenada. A implementação expõe o endereço como inteiro hexadecimal de seis dígitos. Esse endereço é útil para discutir dicionários, eventos esparsos e referências locais a estados.

4 Capacidade e taxas nominais

4.1 Das escolhas aos bits

Para $N = WHF$ módulos selecionáveis independentemente, a quantidade de sequências densas é

$$|\mathcal{D}| = K^N.$$

Uma mensagem binária de comprimento L exige 2^L codificações distinguíveis. Portanto,

$$2^L \le K^N \quad \Longrightarrow \quad L_{\max} = \lfloor WHF \log_2 K \rfloor.$$

No perfil máximo,

$$N = 64^3 = 262.144, \qquad \log_2 K = 6,$$

$$L_{\max} = 1.572.864 \text{ bits}, \qquad B_{\max} = \left\lfloor \frac{L_{\max}}{8} \right\rfloor = 196.608 \text{ bytes} = 192 \text{ KiB}.$$

O payload bruto por estado é

$$B_{\text{state}} = \frac{64^2 \cdot 6}{8} = 3.072 \text{ bytes} = 3 \text{ KiB}.$$

A quantidade de sequências completas possíveis é exponencial em N. O payload binário é seu logaritmo e cresce proporcionalmente a F.

4.2 Taxa do catálogo e taxa de payload

Seja f a quantidade nominal de estados exibidos por segundo. Um ciclo completo tem duração

$$\tau = F/f.$$

Com $F = 64$ e $f = 8$, $\tau = 8$ segundos. O orçamento de percurso do catálogo é

$$R_\Omega = \frac{WHFK}{\tau} = WHKf = 2.097.152 \text{ tuplas candidatas/s}.$$

Essa é a interpretação pretendida se a abreviação **cps** for usada para o catálogo neste RfD. Duas outras taxas descrevem os dados selecionados:

$$R_{\text{modules}} = WHf = 32.768 \text{ módulos/s},$$

$$R_{\text{bits}} = WHf \log_2 K = 196.608 \text{ bits/s} = 24 \text{ KiB/s}.$$

Medida	Perfil máximo
Candidatas de quatro coordenadas	16.777.216 por ciclo
Orçamento de tuplas candidatas	2.097.152 por segundo a 8 fps
Módulos selecionados	262.144 por ciclo
Payload bruto	192 KiB por ciclo
Taxa de payload bruto	24 KiB/s a 8 fps

Para um enquadramento futuro com h bits reservados por estado e taxa de código r, um primeiro orçamento de payload é

$$B_{\text{net}} = \left\lfloor \frac{rF(WH \log_2 K - h)}{8} \right\rfloor, \qquad 0 < r \le 1.$$

A expressão pressupõe reservas fixas por estado, seguidas de um código agregado com taxa r. Outra posição das reservas resulta em outro orçamento. O arranjo eventual é um item de discussão.

5 Tempo no contêiner e tempo no app

5.1 Cadência de transporte

Os delays do GIF são armazenados em centésimos de segundo. Um período nominal de 125 ms por estado pode ser representado por delays alternados de 120 e 130 ms. Para uma quantidade par de frames,

$$\sum_{i=0}^{F-1} \delta_i = 125F \text{ ms}.$$

O protótipo quantiza os limites cumulativos:

$$\delta_i = 10 \left(\left\lfloor \frac{(i+1)p}{10} \right\rfloor - \left\lfloor \frac{ip}{10} \right\rfloor \right) \text{ ms},$$

onde p é o período solicitado em milissegundos. Para $p = 125$ e $F = 64$, os delays armazenados somam 8.000 ms.

5.2 Sequência mantida no app e seleção

A interação proposta coloca a sequência no app. O usuário ou um serviço receptor seleciona ou consome um estado. O app pode avançar em um intervalo escolhido Δ, inclusive um estado por minuto.

Um seletor possível de estado é

$$i(\theta) = \left\lfloor \frac{\theta - \theta_0}{\Delta} \right\rfloor \bmod F.$$

Se o usuário selecionar no instante θ_c, o índice congelado será

$$i_c = i(\theta_c).$$

Um mapeamento escolhido de estado para código, g, fornece o valor para a área de transferência:

$$c = g(D_{i_c}).$$

O visualizador local de estudo usa a string hexadecimal canônica do payload do estado como g. Inicia com $\Delta = 60$ segundos. Um ciclo de 64 estados dura, então, 64 minutos. O exemplo de transporte a 8 fps e a interação de um frame por minuto são dois usos dos mesmos estados ordenados.

Evento	Ação do app	Lado receptor
Carregar	Mantém a sequência e seu mapeamento localmente	Aguarda um código
Avançar	Exibe o próximo estado	Observa ou solicita seleção
Clicar	Interrompe o avanço e congela o estado exibido	Usuário seleciona o estado atual
Copiar	Copia o código hexadecimal associado	Usuário, serviço ou site o consome
Continuar	Retoma quando solicitado	Outro estado pode ser selecionado

O exemplo renderiza cada estado selecionado como um GIF independente. O clique cancela o temporizador do app antes de escrever na área de transferência. Assim, a imagem selecionada e o código copiado se referem ao mesmo estado.

5.3 Relação com TOTP

A comparação com TOTP diz respeito à posse do estado evolutivo e ao papel do verificador receptor. No TOTP, um app mantém um segredo e calcula um código usando um contador temporal. Nesta proposta, o app mantém ou obtém uma sequência e expõe o código de um estado selecionado.

O serviço receptor é o consumidor ou verificador nos dois padrões de interação. O RfD convida à discussão sobre a escolha de g, provisionamento, renovação dos estados e regras de aceitação para esse modelo de sequência. A Clipboard API fornece a operação de cópia acionada pelo usuário utilizada no exemplo local.

6 Unicode e mapeamentos hexadecimais um a um

6.1 O domínio escalar

O domínio escalar Unicode é

$$\mathcal{U} = \{0, \ldots, \texttt{0x10FFFF}\} \setminus \{\texttt{0xD800}, \ldots, \texttt{0xDFFF}\}.$$

Sua cardinalidade é

$$|\mathcal{U}| = \texttt{0x110000} - \texttt{0x800} = 1.112.064.$$

Os “aproximadamente um milhão de códigos” da proposta se relacionam naturalmente com esse domínio. UTF-8 é uma codificação de comprimento variável desses escalares, usando de um a quatro octetos.

Uma representação hexadecimal direta é

$$h(u) = \mathrm{hex}_6(u).$$

Ela é injetiva: $h(u_1) = h(u_2)$ implica $u_1 = u_2$. A representação cabe no domínio de valores de 24 bits já usado pelo catálogo e pelas descrições da paleta.

6.2 Três interpretações candidatas

Dicionário direto de paleta. Um escalar pode ser associado a uma entrada hexadecimal de seis dígitos. Um frame pode utilizar até K entradas escalares distintas nessa interpretação. Um dicionário adaptativo poderia alterar as entradas ativas entre estados. Usos repetidos de uma entrada continuam como escolhas repetidas comuns de módulo.

Representação escalar em quatro dimensions. Um alfabeto fixo de 64 identificadores de dimension pode representar qualquer escalar usando quatro dígitos na base 64:

$$u = 64^3 d_0 + 64^2 d_1 + 64 d_2 + d_3.$$

Os dígitos são

$$d_j = \left\lfloor \frac{u}{64^{3-j}} \right\rfloor \bmod 64, \qquad j = 0, 1, 2, 3.$$

Como

$$64^3 < |\mathcal{U}| < 64^4,$$

quatro dígitos de comprimento fixo na base 64 são suficientes, enquanto três não conseguem identificar todos os escalares. O protótipo oferece esse mapeamento reversível.

Dicionário de endereços do catálogo. O inteiro do escalar também pode ser inserido no domínio de endereços de 24 bits do catálogo. As equações inversas de endereço produzem uma tupla de coordenadas. Isso permite uma entrada de dicionário um a um ou um evento endereçado. Uma sequência de eventos endereçados também precisa de uma representação da ordem e da multiplicidade; essas passam a ser escolhas explícitas de projeto.

Exemplo de escalar	Escalar hexadecimal	Octetos UTF-8	Quatro identificadores de dimension
A latino maiúsculo	`0x000041`	`41`	`00 00 01 01`
a latino minúsculo com til	`0x0000E3`	`C3 A3`	`00 00 03 23`
Sinal de euro	`0x0020AC`	`E2 82 AC`	`00 02 02 2C`
Rosto sorridente	`0x01F600`	`F0 9F 98 80`	`00 1F 18 00`

6.3 Orçamento de texto e eventual chave

A representação em quatro dimensions usa 24 bits por escalar:

$$B_{\text{scalar}} = 3 \text{ bytes}, \qquad C_{\text{state}} = \left\lfloor \frac{4096}{4} \right\rfloor = 1024 \text{ escalares}.$$

Um ciclo máximo de 64 estados comporta 65.536 dessas representações escalares de comprimento fixo, antes do enquadramento. Em comparação com UTF-8, o mapeamento escalar gasta mais bits com valores de um octeto e menos com valores de quatro octetos. A comparação útil depende, portanto, do repertório textual e do projeto do dicionário.

O RfD deixa o termo "chave" aberto a uma interpretação concreta: dicionário público, dicionário privado provisionado, índice de estado externo ou entrada de uma construção criptográfica. O mapeamento um a um estabelece uma representação; seu eventual papel poderá ser escolhido na discussão.

7 Estudo reproduzível e exemplo do app

7.1 O estudo de 16 estados

A demonstração publicada contém deliberadamente 16 estados. Cada estado usa o canvas atual de 256 × 256 pixels, 64 × 64 módulos efetivos e 64 entradas hexadecimais na paleta. A paleta ordenada é rotacionada entre estados. O payload é sintético e reproduzível a partir da semente 20261104.

Seu orçamento bruto de dados é

$$B_{16} = \frac{64 \cdot 64 \cdot 16 \cdot 6}{8} = 49.152 \text{ bytes} = 48 \text{ KiB}.$$

O decoder recupera exatamente o payload original. O relatório registra os valores SHA-256 original e recuperado, o hash do GIF, versões do ambiente, paletas hexadecimais, quantidade de estados e duração nominal.

Propriedade	Estudo medido
Imagem armazenada	256 × 256 pixels
Estado efetivo	64 × 64 módulos
Frames	16
Entradas da paleta por estado	64
Ciclo nominal armazenado	2.000 ms
Payload recuperado	49.152 bytes
Tamanho do GIF codificado	383.644 bytes
Recuperação	Exata, byte a byte

O tamanho do arquivo codificado é uma medida do contêiner. A contagem do payload bruto vem da quantidade de dimensions independentes selecionadas.

7.2 Implementação e verificações

O código aceita octetos de payload em hexadecimal e entradas hexadecimais de paleta. Escreve frames indexados completos, inclusive estados repetidos, e lê os índices armazenados na ordem declarada pela paleta. O decoder verifica que cada módulo efetivo contém 4 × 4 índices idênticos.

A suíte de testes verifica:

- Recuperação exata para diferentes comprimentos de payload, incluindo octetos zero.
- Preservação da ordem variável da paleta.
- Registros de frame distintos para estados idênticos.
- O limite do perfil de 64 frames e 192 KiB.
- Cadência de um estado por minuto.
- Limites de frames, paletas e entrada hexadecimal.
- Rejeição de contêiner truncado e de módulo inconsistente.
- Inversão dos endereços de quatro coordenadas e fronteiras dos escalares Unicode.

O visualizador local inicia em inglês, oferece português e pausa antes de copiar o estado atual. Inclui cópia manual quando a operação direta da área de transferência está indisponível. O código completo copiado no exemplo contém 6.144 dígitos hexadecimais para um estado de 3 KiB.

O repositório fornece o gerador do estudo, resultados, visualizador, fontes dos dois manuscritos e comandos de compilação.

8 Escopo e limitações

Esta seção concentra a interpretação e as fronteiras do RfD.

Status de discussão. Perfil, vocabulário e mapeamentos são propostas de trabalho. Enquadramento, comprimento do código, interoperabilidade, provisionamento e protocolo de verificação são temas de discussão antes de uma especificação.

Dimensões lógicas e contagem. Os quatro eixos enumeram tuplas candidatas. Uma dimension é selecionada por posição espacial e temporal. As 16.777.216 entradas do catálogo e os 192 KiB de payload bruto selecionado medem objetos distintos. “cps”, quando empregado, designa tuplas candidatas por segundo.

Interface hexadecimal e contêiner. Hexadecimal é a representação pública dos valores da paleta e dos códigos. GIF89a fornece sua serialização interna. Mudar a representação numérica não cria entropia adicional. O adaptador atual usa frames completos e opacos e paletas cujo tamanho é potência de dois, com até 64 entradas. O comprimento original do payload é fornecido pelo chamador.

Orçamentos e tempo. Cabeçalhos, dicionários, dados de autenticação, redundância e armazenamento do contêiner exigem orçamentos separados. O exemplo de 8 fps descreve tempo nominal de reprodução; a cadência dos estados do app é independente. O estudo não mede velocidade de rede, vazão de modelos generativos nem captura física.

Autenticação. O exemplo do app contém estados sintéticos públicos e demonstra seleção e cópia. Uma sequência finita repetida exige um projeto escolhido de provisionamento e renovação para uso em autenticação. Confidencialidade, vínculo com desafio, expiração, tratamento de replay e limitação de tentativas pertencem a esse projeto. TOTP e AEAD são referências para a discussão.

Unicode e dicionários. Valores escalares, caracteres atribuídos, clusters de grafemas, octetos UTF-8, entradas ativas de paleta e chaves possuem domínios diferentes. Os mapeamentos escalares preservam a identidade escalar; normalização, ordem, multiplicidade e armazenamento do dicionário exigem escolhas explícitas em um projeto de texto ou chave.

Evidência. Os resultados registrados se aplicam ao caminho atual de arquivo indexado exato. Sustentam as contagens e as alegações de recuperação declaradas e não estabelecem a segurança de um protocolo posterior.

9 Questões para discussão

1. Quais aplicações se beneficiam do catálogo de quatro coordenadas: dados densos, eventos endereçados, seleção de estados ou consulta a dicionário?
2. Qual objeto hexadecimal deve ser chamado de chave e como deverá ser provisionado?
3. O texto deve usar octetos UTF-8, valores escalares em quatro dimensions ou dicionários adaptativos por estado?
4. Como associar o código selecionado a um serviço, desafio ou janela de aceitação?
5. Quais aplicações precisam de transporte rápido, cadência lenta no app ou avanço explícito pelo usuário?
6. O que deve renovar uma sequência após um ciclo ou após o consumo?
7. Quais campos são necessários antes da proposta de uma especificação?

10 Autoria e assistência

João Paulo De Luca Jr e Jorge Luiz Espanhol são os coautores deste RfD e da proposta de discussão associada. ChatGPT/Codex, OpenAI, auxiliou na exposição matemática, tradução, implementação, execução e preparação do documento. Os autores mantêm a responsabilidade pela direção e pelas decisões de pesquisa.

11 Referências técnicas com links

1. CompuServe, Graphics Interchange Format Version 89a, reproduzido pelo W3C.
2. Pillow, suporte a arquivos GIF.
3. Unicode 18.0, Capítulo 3: Conformidade, especialmente §3.9.
4. RFC 3629: UTF-8, a transformation format of ISO 10646.
5. RFC 6238: TOTP, Time-Based One-Time Password Algorithm.
6. RFC 5116: An Interface and Algorithms for Authenticated Encryption.
7. W3C, Clipboard API and events.